

Human Digital Twin for Behavior Simulation Using Artificial Intelligence

Open Research Project

Yenni Vineeth Kumar

Department of Computer Science and Engineering
Krishna University College of Engineering and Technology
Machilipatnam, Andhra Pradesh, India
vineethyenni@gmail.com

Artificial Intelligence
Machine Learning
Natural Language Processing
Emotion Classification
Behavioral Simulation
Human Digital Twin Framework

Abstract

Modeling human behavior within artificial intelligence systems remains a complex challenge due to the dynamic interplay between emotional states and contextual factors, where existing approaches often focus on isolated aspects such as emotion recognition or context-based reasoning, resulting in limited capability to simulate realistic human decision-making. This project presents a Human Digital Twin (HDT) framework designed to simulate behavior by integrating emotion detection with contextual scenario modeling, where user-provided textual input is processed using Term Frequency–Inverse Document Frequency (TF-IDF) for feature extraction, followed by a Logistic Regression model to identify the dominant emotional state. To enhance behavioral simulation, contextual parameters such as scenario type and difficulty level are incorporated alongside emotional features, and these combined inputs are used to predict decision tendencies, enabling the system to model how individuals respond under varying conditions. In the absence of publicly available datasets that jointly represent emotion, context, and behavior, a structured synthetic dataset is constructed to support controlled training and evaluation of the behavioral prediction model. The system is implemented using Python-based tools and provides an interactive interface for real-time simulation, and in addition to prediction, it incorporates an explanation module and a behavioral interpretation layer, thereby improving transparency and enabling a clearer understanding of decision patterns. Experimental evaluation using both controlled and real-world data demonstrates that the system is capable of generating consistent and meaningful behavioral predictions while reflecting the inherent complexity of human emotional expression, and the proposed framework emphasizes interpretability and realistic behavior modeling, making it suitable for applications in decision support systems, human-computer interaction, and intelligent simulation environments.

Contents

Abstract	1
Abbreviations	7
1 Introduction	8
2 Background Concepts	10
2.1 Digital Twin	10
2.2 Human Digital Twin	10
2.3 Natural Language Processing	10
2.4 Feature Extraction using TF-IDF	11
2.5 Logistic Regression	11
2.6 Emotion Classification	11
2.7 Contextual Modeling	12
2.8 Behavioral Prediction	12
2.9 Interpretability in AI Systems	12
3 Literature Survey	13
4 Problem Statement and Objectives	15
4.1 Problem Statement	15
4.2 Objectives	15
5 Existing System Analysis	17
6 Proposed System	18
7 System Architecture	20
8 Methodology	24
8.1 Input Acquisition	24
8.2 Text Preprocessing	24
8.3 Feature Extraction using TF-IDF	24
8.4 Emotion Classification	24
8.5 Contextual Parameter Integration	25
8.6 Feature Integration	25
8.7 Behavior Prediction Model	25

8.8	Explanation Generation	25
8.9	Behavioral Interpretation	25
8.10	Final Output Generation	26
9	Dataset Description	27
9.1	GoEmotions Dataset	27
9.2	Need for Synthetic Dataset	27
9.3	Synthetic Dataset Design	27
9.4	Behavioral Outcome Representation	29
9.5	Data Structure	29
9.6	Sample Data Illustration	29
9.7	Dataset Usage in the System	29
10	Implementation Details	31
10.1	Development Environment	31
10.2	Synthetic Dataset Generation	31
10.3	Feature Encoding and Preprocessing	31
10.4	Feature Scaling	32
10.5	Behavior Prediction Model	32
10.6	GoEmotions Data Processing	32
10.7	TF-IDF Feature Extraction	32
10.8	Emotion Classification Model	33
10.9	Simulation Engine	33
10.10	Behavioral Interpretation	33
10.11	User Interface (Streamlit)	33
10.12	Evaluation Module	34
10.13	End-to-End Simulation Pipeline	34
11	Results and Discussion	35
11.1	Synthetic Dataset Evaluation	35
11.2	GoEmotions Dataset Evaluation	37
11.3	Comparative Analysis	39
11.4	Key Observations	39
11.5	Discussion	40
12	Real-Time Simulation	41
12.1	Overview of Real-Time Simulation	41
12.2	User Interface Design	41
12.3	Scenario Selection Mechanism	42
12.4	Input Processing Example	43

12.5	Simulation Execution	43
12.6	Behavioral Simulation Output	44
12.7	Emotion Confidence Visualization	44
12.8	Decision Prediction Visualization	45
12.9	Emotion to Behavior Mapping	45
12.10	Explanation Module	46
12.11	Behavioral Interpretation Module	46
12.12	End-to-End Simulation Flow	46
12.13	System Characteristics	47
12.14	Summary	47
13	Advantages	48
13.1	Integration of Emotion and Context	48
13.2	Realistic Behavioral Simulation	48
13.3	Use of Synthetic Dataset for Controlled Learning	48
13.4	High Accuracy in Behavioral Prediction	49
13.5	Multi-Label Emotion Detection Capability	49
13.6	Interpretability and Explanation Module	49
13.7	Behavioral Interpretation Layer	49
13.8	Real-Time Interactive Simulation	50
13.9	Scalable and Modular Architecture	50
13.10	Flexibility Across Multiple Domains	50
13.11	Improved Decision Understanding	50
13.12	Summary	50
14	Limitations	51
14.1	Dependence on Synthetic Behavioral Dataset	51
14.2	Limited Availability of Real-World Behavioral Data	51
14.3	Imbalance in GoEmotions Dataset	51
14.4	Lower Accuracy in Emotion Classification	51
14.5	Limited Emotional Coverage	52
14.6	Limited Behavioral Mapping	52
14.7	Simplified Decision Modeling	52
14.8	Limited Contextual Understanding	52
14.9	Static Scenario Categories	52
14.10	Absence of Advanced Explainability Techniques	53
14.11	Static Model Behavior	53
14.12	Limited Personalization	53

14.13	Scalability to Complex Real-World Scenarios	53
14.14	Summary	53
15	Future Work	54
15.1	Expansion of Emotion–Behavior Mapping	54
15.2	Incorporation of Real-World Behavioral Data	54
15.3	Multi-Turn Context Analysis	54
15.4	Adaptive Behavioral Learning	54
15.5	Personalized Digital Twin Modeling	55
15.6	Integration of Advanced Explainability Techniques	55
15.7	Enhanced Contextual Modeling	55
15.8	Dynamic Scenario Generation	55
15.9	Integration with Deep Learning Models	55
15.10	Real-Time Continuous Simulation	56
15.11	Cross-Domain Applications	56
15.12	Deployment and Scalability Enhancements	56
15.13	Summary	56
16	Conclusion	57
	References	59
	Appendix	60

List of Figures

1	Human Digital Twin System Architecture	21
2	Confusion Matrix for Synthetic Behavior Prediction Model	35
3	Performance Metrics for Synthetic Model	36
4	Confusion Matrix for GoEmotions Emotion Classification Model	37
5	Performance Metrics for GoEmotions Model	38
6	HDT System Main Interface	41
7	Scenario Selection Options	42
8	User Input with Scenario and Difficulty Selection	43
9	Behavioral Simulation Result	44
10	Emotion Confidence Visualization	44
11	Predicted Decision Visualization	45
12	Emotion to Behavior Mapping and Interpretation	45

Abbreviations

- HDT – Human Digital Twin
- AI – Artificial Intelligence
- ML – Machine Learning
- NLP – Natural Language Processing
- TF-IDF – Term Frequency–Inverse Document Frequency
- UI – User Interface

1 Introduction

Understanding human behavior within artificial intelligence systems is a complex challenge due to the continuous interaction between emotional states and contextual conditions. Human decisions are not driven by a single factor; instead, they are influenced by a combination of internal emotions and external situations. Capturing this interaction in a computational model remains a significant limitation in existing artificial intelligence approaches.

Most traditional systems in this domain focus on isolated aspects of behavior modeling. Emotion recognition techniques analyze textual or sensory inputs to classify emotional states, while context-based systems attempt to model decisions based on environmental conditions or predefined rules. Although these approaches provide useful insights, they operate independently and fail to capture the combined influence of emotional and contextual factors on decision-making.

In real-world scenarios, the same situation can produce different decisions depending on an individual’s emotional state, and similarly, the same emotional condition can lead to different outcomes under varying contexts. This dependency highlights the need for a unified framework that integrates both emotional and contextual dimensions to simulate realistic human behavior.

The concept of a Digital Twin, originally developed for modeling physical systems, provides a strong foundation for addressing this problem. A Digital Twin represents a virtual replica of a real-world entity, enabling analysis and prediction under different conditions. Extending this concept to human behavior leads to the development of a Human Digital Twin (HDT), where decision-making patterns can be simulated by incorporating psychological and situational factors.

This project proposes a Human Digital Twin framework designed to simulate behavior by combining emotion detection with contextual scenario modeling. The system processes user-provided textual input and transforms it into numerical features using Term Frequency–Inverse Document Frequency (TF-IDF). A Logistic Regression model is used to identify the dominant emotional state associated with the input.

To improve the realism of behavioral simulation, contextual parameters such as scenario type and difficulty level are integrated with emotional features. These combined inputs enable the system to predict decision tendencies, reflecting how individuals respond under different conditions. Since publicly available datasets do not provide a direct mapping between emotion, context, and behavior, a synthetic dataset is constructed to support controlled training and evaluation.

In addition to prediction, the system emphasizes interpretability by incorporating an

explanation module and a behavioral interpretation layer. These components provide insights into how emotional and contextual factors influence the predicted outcome, making the system more transparent and understandable.

The proposed framework aims to bridge the gap between isolated emotion analysis and context-based reasoning by providing an integrated approach to behavior simulation. This enables more realistic modeling of human decision-making and supports applications in areas such as decision support systems, human-computer interaction, and intelligent simulation environments.

2 Background Concepts

Understanding the proposed Human Digital Twin (HDT) framework requires a clear foundation of several key concepts from artificial intelligence, natural language processing, and behavior modeling. These concepts provide the theoretical basis for designing a system that integrates emotional analysis with contextual understanding to simulate human decision-making.

2.1 Digital Twin

The concept of a Digital Twin originated in the field of engineering as a virtual representation of a physical system. It enables real-time monitoring, analysis, and prediction of system behavior under different conditions. A Digital Twin continuously receives data from its real-world counterpart and updates its state accordingly, allowing simulation and optimization without directly interacting with the physical entity.

This concept has gradually extended beyond physical systems into more abstract domains, including human-centric modeling. When applied to human behavior, it evolves into a Human Digital Twin, where the goal is to replicate and simulate decision-making patterns based on internal and external influences.

2.2 Human Digital Twin

A Human Digital Twin (HDT) represents a computational model that simulates human behavior by capturing key influencing factors such as emotional states, contextual conditions, and decision tendencies. Unlike traditional systems that focus on isolated attributes, an HDT aims to integrate multiple dimensions of human behavior into a unified framework.

The primary objective of an HDT is not only to analyze behavior but also to predict how an individual might respond under varying conditions. This makes it particularly useful in applications such as decision support systems, personalized interfaces, and intelligent simulations.

2.3 Natural Language Processing

Natural Language Processing (NLP) is a branch of artificial intelligence that focuses on enabling machines to understand and process human language. In this project, NLP is used to analyze textual input provided by the user and extract meaningful features that represent underlying emotional states.

Text data is inherently unstructured, making it necessary to convert it into a structured format before applying machine learning models. This transformation is achieved through preprocessing and feature extraction techniques, which allow the system to interpret language in a computational form.

2.4 Feature Extraction using TF-IDF

Term Frequency–Inverse Document Frequency (TF-IDF) is a widely used technique for converting textual data into numerical representations. It assigns weights to words based on their importance within a document relative to a collection of documents.

The term frequency component measures how often a word appears in a given text, while the inverse document frequency reduces the weight of commonly occurring words across multiple texts. This combination helps in identifying words that carry meaningful information for classification tasks.

In this project, TF-IDF is used to transform user input into feature vectors that can be processed by the machine learning model for emotion classification.

2.5 Logistic Regression

Logistic Regression is a supervised machine learning algorithm used for classification tasks. It estimates the probability of a given input belonging to a particular class based on a linear combination of input features.

Although it is a relatively simple model, Logistic Regression is effective for text classification tasks when combined with appropriate feature extraction methods such as TF-IDF. Its simplicity also contributes to better interpretability, which aligns with the objective of the proposed system.

2.6 Emotion Classification

Emotion classification involves identifying the emotional state expressed in textual input. In this system, the classified emotion serves as a key factor influencing behavioral prediction.

The model analyzes textual features extracted through TF-IDF and predicts the dominant emotion associated with the input. This emotional representation is later combined with contextual parameters to simulate decision-making behavior.

2.7 Contextual Modeling

Contextual modeling refers to the representation of external factors that influence human decisions. These factors may include the type of situation, difficulty level, or environmental conditions.

In real-world scenarios, behavior is rarely determined by emotion alone. The same emotional state can lead to different outcomes depending on the context. Therefore, incorporating contextual parameters is essential for realistic behavior simulation.

2.8 Behavioral Prediction

Behavioral prediction is the process of estimating decision outcomes based on a combination of emotional and contextual inputs. In the proposed system, this is achieved by integrating features derived from emotion classification with contextual parameters.

The model learns patterns from the synthetic dataset and uses them to predict how an individual is likely to respond under specific conditions. This step represents the core functionality of the Human Digital Twin framework.

2.9 Interpretability in AI Systems

Interpretability refers to the ability of a system to explain its predictions in a human-understandable manner. Many modern machine learning models act as black boxes, providing results without clear reasoning.

In this project, interpretability is emphasized by including an explanation module and a behavioral interpretation layer. These components help in understanding how emotional and contextual factors contribute to the predicted outcome, making the system more transparent and reliable.

3 Literature Survey

The concept of Digital Twin has been widely explored in the field of engineering and industrial systems to model, monitor, and predict the behavior of complex physical entities. Grieves and Vickers introduced the Digital Twin as a virtual representation of a real-world system that enables analysis of system behavior under varying conditions [1]. This concept has been further extended by Tao et al., who highlighted its applications in smart manufacturing and real-time system optimization [2]. While these approaches focus on physical systems, they provide a strong foundation for extending Digital Twin technology to human-centric domains.

In the context of human behavior modeling, traditional approaches primarily focus on emotion recognition or context-based decision systems independently. Emotion recognition techniques leverage Natural Language Processing (NLP) and machine learning models to classify emotional states from textual data. The GoEmotions dataset, introduced by Demszky et al., provides a large-scale dataset with fine-grained emotion categories, enabling more detailed analysis of emotional expressions [3]. However, such datasets are designed mainly for classification tasks and do not directly address behavior prediction.

On the other hand, context-aware systems attempt to simulate decision-making based on environmental or situational factors. These systems often rely on predefined rules or structured parameters, which limit their flexibility and adaptability. Moreover, they do not incorporate internal emotional states, which play a significant role in influencing human decisions.

Recent research has also emphasized the importance of human-centered artificial intelligence, where systems are designed to be transparent, reliable, and interpretable. Shneiderman highlighted the need for AI systems that not only produce accurate predictions but also provide understandable explanations for their outputs [4]. This aspect is particularly important in behavior modeling, where interpretability is essential for trust and usability.

Despite these advancements, existing approaches suffer from a key limitation: the lack of integration between emotional and contextual factors. Emotion-based systems do not consider situational influence, while context-based systems ignore emotional variability. Additionally, most existing works focus on static prediction rather than dynamic simulation of behavior.

Therefore, there is a clear research gap in developing a unified framework that combines emotional analysis with contextual modeling to simulate realistic human decision-making. The proposed Human Digital Twin (HDT) framework addresses this gap by integrating emotion detection, contextual parameters, and behavioral prediction into a single system.

Furthermore, the inclusion of interpretability components ensures that the system not only predicts behavior but also explains the reasoning behind its decisions.

4 Problem Statement and Objectives

4.1 Problem Statement

Modeling human behavior in artificial intelligence systems is a challenging task due to the complex interaction between emotional states and contextual factors. Most existing approaches focus on either emotion recognition or context-based decision modeling independently. While emotion recognition systems analyze textual or sensory data to classify emotional states, context-based systems rely on predefined rules or environmental parameters to simulate decisions.

However, human decision-making in real-world scenarios is influenced by both internal emotions and external conditions simultaneously. The same situation can result in different decisions depending on an individual's emotional state, and similarly, the same emotional condition can lead to varying outcomes under different contexts. This dynamic interaction is not effectively captured by existing systems.

Additionally, many current models lack interpretability, making it difficult to understand the reasoning behind predicted outcomes. This limits their usability in applications where transparency and trust are essential.

Therefore, there is a need for a unified system that integrates emotional analysis with contextual modeling to simulate realistic human decision-making behavior while maintaining interpretability.

4.2 Objectives

The primary objective of this project is to develop a Human Digital Twin (HDT) framework capable of simulating human behavior by combining emotional and contextual information. The specific objectives are as follows:

- To design a system that processes textual input and identifies the underlying emotional state using Natural Language Processing techniques.
- To implement feature extraction using TF-IDF for converting textual data into a structured numerical format.
- To develop an emotion classification model using Logistic Regression for accurate identification of emotional states.
- To incorporate contextual parameters such as scenario type and difficulty level for realistic behavior modeling.

- To create a synthetic dataset that represents the relationship between emotional states, contextual factors, and behavioral outcomes.
- To design a behavioral prediction mechanism that estimates decision tendencies based on combined emotional and contextual inputs.
- To implement an explanation module that provides reasoning behind predicted decisions, improving system interpretability.
- To develop an interactive interface that enables real-time simulation of human decision-making behavior.

5 Existing System Analysis

Existing approaches for modeling human behavior in artificial intelligence primarily focus on individual aspects such as emotion recognition or context-based decision systems. While these approaches provide partial understanding, they fail to capture the combined influence of emotional and contextual factors, which is essential for realistic behavior simulation.

Emotion recognition systems analyze textual, facial, or speech data to identify emotional states using techniques such as Natural Language Processing and machine learning models. In this project context, such systems can classify emotions from user input using methods like TF-IDF and Logistic Regression. However, these systems are limited to identifying emotions and do not extend to predicting decision-making behavior based on those emotions.

On the other hand, context-based systems attempt to model decisions by considering external factors such as environment, task complexity, or predefined conditions. These systems often rely on rule-based logic or structured parameters. Although they simulate decision-making to some extent, they ignore the internal emotional state of individuals, which plays a critical role in influencing behavior.

In addition, many existing systems lack integration between emotional analysis and contextual modeling. This separation results in incomplete representations of human behavior, as real-world decisions are influenced by both internal and external factors simultaneously. Furthermore, most systems are designed for static prediction rather than dynamic simulation, limiting their ability to represent varying behavioral responses across different scenarios.

Another important limitation observed in existing approaches is the lack of interpretability. Many machine learning models provide predictions without clearly explaining the reasoning behind them, making it difficult to understand how different factors contribute to the final outcome. This reduces the reliability and practical usability of such systems.

In summary, existing systems in the context of this project are limited by their isolated approach to emotion or context, lack of integration between key influencing factors, restricted capability in simulating realistic behavior, and absence of clear interpretability. These limitations highlight the need for a unified framework that combines emotional and contextual information to enable accurate and transparent behavior simulation, which is addressed in the proposed Human Digital Twin system.

6 Proposed System

The proposed system introduces a Human Digital Twin (HDT) framework designed to simulate human decision-making behavior by integrating emotional analysis with contextual scenario modeling. Unlike existing approaches that focus on isolated aspects, the proposed system provides a unified structure where both internal emotional states and external situational factors are considered simultaneously.

The primary objective of the system is to model how individuals respond to different situations by analyzing textual input and combining it with contextual parameters. The system is designed as a modular pipeline, where each component performs a specific function while contributing to the overall behavioral prediction process.

The workflow begins with user-provided textual input, which represents a situation, thought, or response. This input is processed and converted into a structured numerical format using Term Frequency–Inverse Document Frequency (TF-IDF). The transformed features are then used by a Logistic Regression model to classify the dominant emotional state associated with the input.

To enhance realism in behavior simulation, the system incorporates contextual parameters such as scenario type and difficulty level. These contextual attributes represent external influences that affect decision-making. By combining emotional features with contextual information, the system creates a comprehensive representation of the user’s state.

The integrated feature set is then passed to the behavioral prediction module, which estimates decision tendencies based on learned patterns. This module is trained using a synthetic dataset specifically designed to model the relationship between emotional states, contextual conditions, and behavioral outcomes. The use of a synthetic dataset enables controlled experimentation and allows the system to learn structured relationships that are not readily available in real-world datasets.

An important aspect of the proposed system is its focus on interpretability. Unlike traditional black-box models, the system includes an explanation module that provides reasoning behind predicted decisions. Additionally, a behavioral interpretation layer translates predictions into meaningful insights, making the system more transparent and user-friendly.

The system is implemented using Python-based tools and provides an interactive interface for real-time simulation. Users can input text and select contextual conditions to observe predicted decisions along with explanations. This interactive capability allows dynamic analysis of behavior under varying scenarios.

Overall, the proposed Human Digital Twin framework addresses the limitations of existing systems by integrating emotional and contextual factors, enabling realistic behavior

simulation, and providing interpretable outputs. This makes the system suitable for applications in decision support systems, human-computer interaction, and intelligent simulation environments.

7 System Architecture

The system architecture of the proposed Human Digital Twin (HDT) framework is designed as a structured pipeline that processes textual input, extracts emotional features, integrates contextual information, and generates behavior predictions along with interpretability outputs. Each component in the architecture is functionally independent yet interconnected, ensuring a smooth flow of data from input to final output.

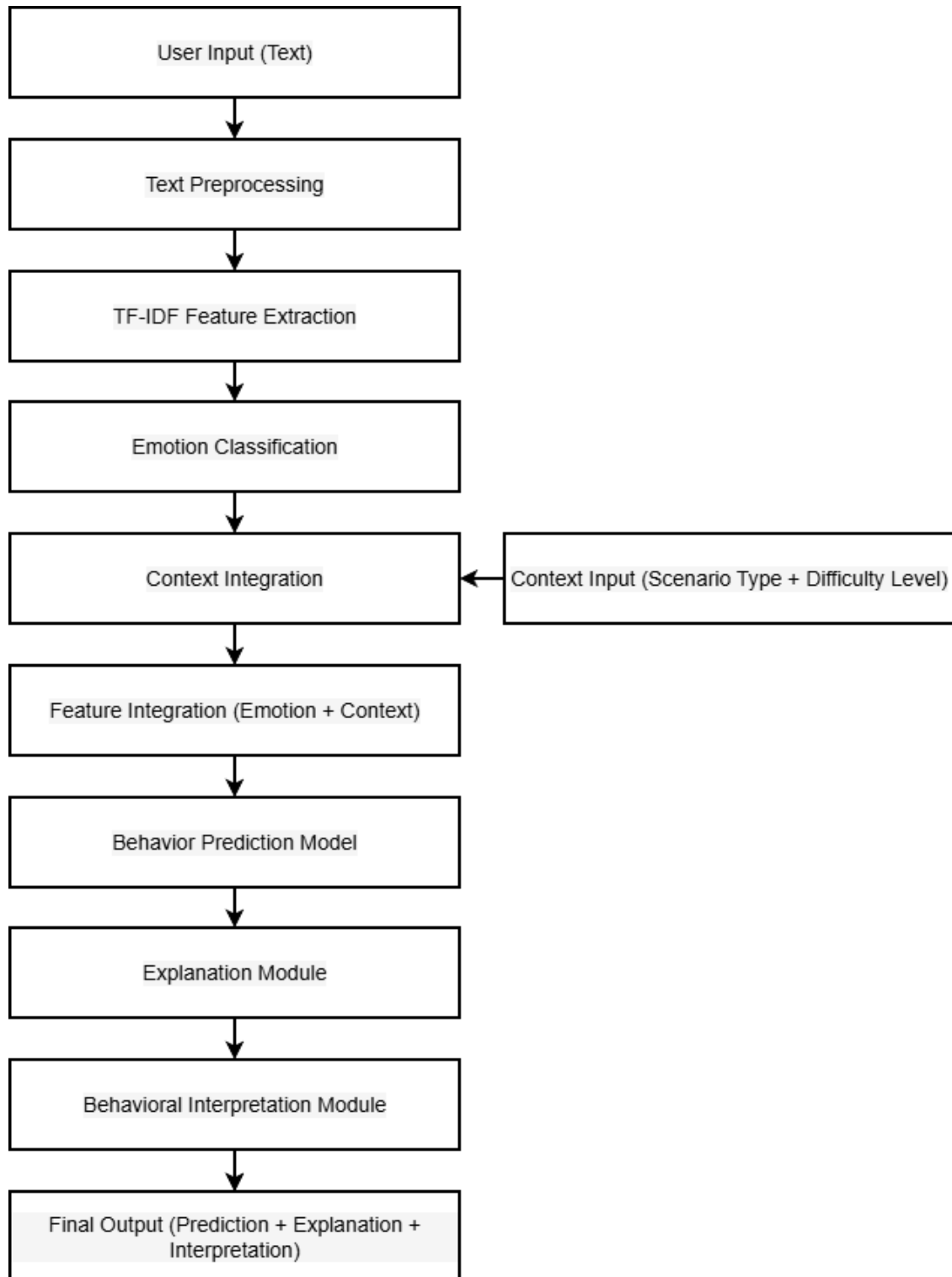


Figure 1: Human Digital Twin System Architecture

The architecture begins with the **User Input (Text)** module, where the user provides textual data representing thoughts, situations, or responses. This raw input forms the pri-

mary source for emotion analysis.

The input text is then passed to the **Text Preprocessing** stage. In this stage, the text is cleaned and standardized to remove noise and inconsistencies. Operations such as lowercasing, removal of stop words, punctuation handling, and tokenization are performed to prepare the text for feature extraction.

Following preprocessing, the cleaned text is transformed into numerical form using the **TF-IDF Feature Extraction** module. This module converts textual data into feature vectors by assigning importance weights to words based on their frequency and relevance. The resulting vector representation enables machine learning models to process textual information effectively.

The extracted features are then provided to the **Emotion Classification** module. This module utilizes a Logistic Regression model to classify the dominant emotional state associated with the input text. The output of this stage is a structured emotional representation that captures the internal state of the user.

Parallel to this process, contextual information is provided through the **Context Input (Scenario Type + Difficulty Level)** module. This module represents external conditions that influence decision-making. The contextual parameters are fed into the **Context Integration** stage, where they are structured and prepared for combination with emotional features.

The system then performs **Feature Integration (Emotion + Context)**, where emotional outputs and contextual parameters are merged into a unified feature set. This step is crucial as it models the real-world interaction between internal emotions and external conditions, enabling more realistic behavior simulation.

The integrated features are passed to the **Behavior Prediction Model**. This model is trained on a synthetic dataset designed to capture relationships between emotion, context, and decision outcomes. Based on the input features, the model predicts behavioral tendencies or decision responses.

To ensure transparency, the system includes an **Explanation Module**. This module analyzes the contribution of different features and provides reasoning behind the predicted behavior. It helps users understand how emotional and contextual factors influenced the final decision.

Further enhancing interpretability, the **Behavioral Interpretation Module** translates the prediction and explanation into meaningful insights. Instead of presenting raw outputs, this module provides human-understandable interpretations of the system's decision.

Finally, the system produces the **Final Output (Prediction + Explanation + Interpretation)**. This output combines the predicted behavior, reasoning, and interpretation into

a single response, enabling comprehensive understanding of the simulated decision-making process.

Overall, the architecture ensures a logical and systematic flow of data, integrating emotion detection, contextual modeling, prediction, and interpretability within a single framework. This design allows the system to simulate realistic human behavior while maintaining clarity and transparency in its outputs.

8 Methodology

The methodology of the proposed Human Digital Twin (HDT) system defines the step-by-step process followed to simulate human behavior by integrating emotional analysis with contextual information. The system is designed as a sequential pipeline where each stage transforms input data into a more structured and meaningful representation, ultimately leading to behavior prediction and interpretation.

8.1 Input Acquisition

The process begins with user-provided textual input. This input represents a thought, reaction, or situation description given by the user. The textual input serves as the primary source for extracting emotional information, which plays a crucial role in influencing behavioral outcomes.

8.2 Text Preprocessing

The raw textual input is first processed to remove noise and ensure consistency. This stage includes multiple steps such as converting all text to lowercase, removing punctuation, eliminating stop words, and tokenizing the text into individual words or tokens. These operations help in reducing irrelevant variations and preparing the text for feature extraction.

8.3 Feature Extraction using TF-IDF

After preprocessing, the cleaned text is transformed into a numerical format using Term Frequency–Inverse Document Frequency (TF-IDF). This method assigns weights to words based on their importance in the given text relative to the dataset. Words that occur frequently in a specific input but are rare across the dataset receive higher importance.

The TF-IDF representation converts textual data into a vector space model, allowing machine learning algorithms to process and analyze the data effectively.

8.4 Emotion Classification

The numerical feature vectors generated from TF-IDF are used as input to the emotion classification model. A Logistic Regression classifier is employed to identify the dominant emotional state associated with the input text.

The model is trained using labeled data where each text instance corresponds to a specific emotional category. Based on learned patterns, the classifier predicts the most probable

emotion for new input data.

8.5 Contextual Parameter Integration

In addition to emotional information, the system incorporates contextual parameters such as scenario type and difficulty level. These parameters represent external conditions that influence decision-making behavior.

The contextual data is structured and encoded into a numerical format so that it can be combined with emotional features. This ensures compatibility between different types of inputs within the system.

8.6 Feature Integration

The emotional features obtained from the classification model and the contextual parameters are combined into a single unified feature vector. This integration step is critical, as it reflects the real-world interaction between internal emotional states and external situational factors.

The combined feature set provides a more comprehensive representation of the user's condition, enabling accurate behavioral prediction.

8.7 Behavior Prediction Model

The integrated feature vector is fed into the behavior prediction model. This model is trained using a synthetic dataset that captures relationships between emotions, contextual parameters, and corresponding behavioral outcomes.

The model analyzes patterns within the combined features and predicts decision tendencies or responses that simulate human behavior under given conditions.

8.8 Explanation Generation

To improve transparency, the system includes an explanation mechanism that analyzes the contribution of different features in the prediction process. This module identifies how emotional and contextual inputs influence the final output and provides reasoning for the predicted behavior.

8.9 Behavioral Interpretation

The explanation output is further processed by the behavioral interpretation module, which converts technical results into meaningful insights. This step ensures that the system output

is understandable and relevant from a human perspective.

8.10 Final Output Generation

The final stage of the methodology produces a comprehensive output consisting of the predicted behavior, explanation, and interpretation. This output enables users to understand not only what decision is predicted but also why it was predicted and how different factors contributed to the outcome.

Overall, the methodology ensures a systematic transformation of raw input into meaningful behavioral predictions through a structured and interpretable process. Each stage is designed to contribute to realism, accuracy, and clarity in simulating human decision-making behavior.

9 Dataset Description

The dataset used in the proposed Human Digital Twin (HDT) system is a combination of real-world emotional data and a structured synthetic dataset. This dual approach is adopted to effectively model the relationship between emotional states, contextual scenarios, and behavioral outcomes, which is not directly available in a single existing dataset.

9.1 GoEmotions Dataset

For emotion classification, the system utilizes the GoEmotions dataset, which contains a large collection of text samples annotated with fine-grained emotional labels. This dataset enables the model to learn realistic emotional patterns from natural language input.

The dataset consists of short textual expressions associated with multiple emotion categories. For the purpose of this system, these emotions are mapped into a simplified set of emotional states to ensure consistency and effective integration with contextual modeling.

The GoEmotions dataset is used exclusively for training the emotion classification component of the system, ensuring that the emotional predictions are grounded in real-world data.

9.2 Need for Synthetic Dataset

Although the GoEmotions dataset provides emotional labels, it does not include contextual information such as scenario type or difficulty level, nor does it define behavioral outcomes. Since the objective of the proposed system is to simulate decision-making behavior influenced by both emotion and context, an additional dataset is required.

To address this limitation, a synthetic dataset is constructed. This dataset is specifically designed to represent structured relationships between scenario types, difficulty levels, emotional states, and behavioral responses.

9.3 Synthetic Dataset Design

The synthetic dataset is built using predefined scenario categories that represent common real-life decision-making situations. The system considers the following scenario types:

- Career
- Money
- Ethics

- Relationship
- Health
- Risk
- Conflict
- Deadline

Each scenario is associated with a set of difficulty levels. The difficulty levels are not uniform across all scenarios; instead, they are carefully mapped based on the nature of the situation. For example, scenarios such as ethics, health, and deadlines are generally associated with higher difficulty levels, while risk-based scenarios may include a wider range from low to high difficulty.

The defined difficulty levels include:

- Low
- Medium
- High
- Very High

Based on the difficulty level, corresponding emotional states are generated. The emotional mapping reflects increasing intensity, where lower difficulty corresponds to stable emotional conditions, and higher difficulty corresponds to stress-driven emotional states.

- Low → Calm, Focused
- Medium → Concerned, Thinking
- High → Anxious, Stressed
- Very High → Panic, Overwhelmed

This mapping ensures that emotional states are logically consistent with the complexity of the scenario, creating a realistic relationship between context and emotion.

9.4 Behavioral Outcome Representation

The final component of the dataset is the behavioral outcome. For each combination of scenario type, difficulty level, and emotional state, a corresponding behavioral response is assigned.

These behavioral outcomes are designed to reflect expected human decisions under different conditions. The mapping is controlled and consistent, allowing the behavior prediction model to learn meaningful relationships between input features and decision outputs.

9.5 Data Structure

Each instance in the synthetic dataset consists of the following attributes:

- Scenario Type
- Difficulty Level
- Emotional State
- Behavioral Outcome

The dataset is organized in a tabular format, where each row represents a unique combination of these attributes.

9.6 Sample Data Illustration

A simplified representation of the dataset is shown below:

Scenario	Difficulty	Emotion	Behavior
career	high	stressed	cautious_decision
money	medium	concerned	calculated_choice
ethics	very high	overwhelmed	moral_conflict
relationship	medium	thinking	balanced_response
risk	low	calm	confident_action

9.7 Dataset Usage in the System

The GoEmotions dataset is used for training the emotion classification model, while the synthetic dataset is used for training the behavior prediction model. This separation ensures that each component of the system is optimized for its specific function.

The integration of real-world emotional data with structured synthetic behavioral data enables the system to simulate realistic human decision-making while maintaining control over the relationships between different influencing factors.

10 Implementation Details

The implementation of the proposed Human Digital Twin (HDT) system is carried out using a modular approach that integrates data processing, machine learning models, simulation logic, and an interactive user interface. The system is developed using Python and various supporting libraries.

10.1 Development Environment

The system is implemented using Python. Key libraries used include Pandas and NumPy for data handling, Scikit-learn for machine learning models, Joblib for model persistence, and Streamlit for building the interactive user interface.

10.2 Synthetic Dataset Generation

The synthetic dataset is generated to model relationships between scenario type, difficulty level, emotional state, and behavioral outcomes. The dataset is created using controlled mappings to ensure realistic behavior simulation.

```
scenario = random.choice(scenarios)
difficulty = random.choice(difficulty_map[scenario])
emotion = random.choice(emotion_map[difficulty])
```

Additional attributes such as personality, stress level, and context are incorporated to enrich the dataset. The generated dataset is stored and reused for model training.

10.3 Feature Encoding and Preprocessing

The dataset features are separated from the target variable and transformed into numerical format using One-Hot Encoding.

```
encoder = OneHotEncoder(sparse_output=False)
features_encoded = encoder.fit_transform(features)
```

This transformation ensures that categorical variables can be effectively used by machine learning models.

10.4 Feature Scaling

To improve model performance, the encoded features are standardized using a scaling technique.

```
scaler = StandardScaler()
features_scaled = scaler.fit_transform(features_encoded)
```

This step ensures uniform feature distribution and improves convergence during training.

10.5 Behavior Prediction Model

The behavior prediction component is implemented using a gradient boosting-based classifier, which learns complex relationships between emotional and contextual features.

```
model = HistGradientBoostingClassifier(max_iter=300, learning_rate=0.07)
model.fit(X_train, y_train)
```

The trained model predicts decision tendencies based on integrated feature inputs.

10.6 GoEmotions Data Processing

The GoEmotions dataset is preprocessed to clean and normalize textual input before feature extraction.

```
text = re.sub(r"[^a-zA-Z\s]", " ", text.lower())
text = re.sub(r"\s+", " ", text).strip()
```

This ensures that only meaningful textual content is retained for analysis.

10.7 TF-IDF Feature Extraction

Textual data is converted into numerical feature vectors using TF-IDF representation.

```
vectorizer = TfidfVectorizer(max_features=10000)
text_features = vectorizer.fit_transform(texts)
```

This enables the model to capture important linguistic patterns.

10.8 Emotion Classification Model

Emotion classification is performed using a Logistic Regression model in a multi-label setting.

```
model = OneVsRestClassifier(LogisticRegression())
model.fit(X_train, y_train)
```

The model predicts probabilities for different emotions, from which the dominant emotion is selected.

10.9 Simulation Engine

The core functionality of the system is implemented in the simulation engine, which integrates emotion prediction with contextual inputs to generate behavioral outcomes.

```
text_vector = vectorizer.transform([user_text])
emotion_probs = model.predict_proba(text_vector)[0]
dominant_emotion = emotion_names[np.argmax(emotion_probs)]
```

The predicted emotion is mapped and combined with contextual parameters before passing to the behavior prediction model.

10.10 Behavioral Interpretation

The system includes a rule-based interpretation layer that converts predictions into meaningful insights.

```
if mapped_emotion in ["anxious", "stressed"]:
    interpretation.append("Decision avoidance tendency increases")
```

This improves system transparency and usability.

10.11 User Interface (Streamlit)

An interactive interface is developed using Streamlit to allow users to input text and select contextual parameters.

```
user_text = st.text_area("Describe your situation")
scenario = st.selectbox("Select Scenario", scenarios)
```

The interface displays predicted decisions, emotion states, confidence levels, and visualizations for better understanding.

10.12 Evaluation Module

The system performance is evaluated using standard metrics such as accuracy and classification reports.

```
accuracy = accuracy_score(y_true, y_pred)
print(classification_report(y_true, y_pred))
```

Confusion matrices are also used to visualize model performance.

10.13 End-to-End Simulation Pipeline

The overall system integrates all components into a unified pipeline. User input is processed, emotional state is detected, contextual parameters are integrated, and behavioral predictions are generated along with explanation and interpretation.

The complete implementation code, including notebooks and system modules, is provided in the Appendix for reference.

11 Results and Discussion

This section presents a detailed evaluation of the proposed Human Digital Twin (HDT) system using both synthetic and real-world datasets. The analysis focuses on model performance, behavioral consistency, and the impact of dataset characteristics on prediction quality.

11.1 Synthetic Dataset Evaluation

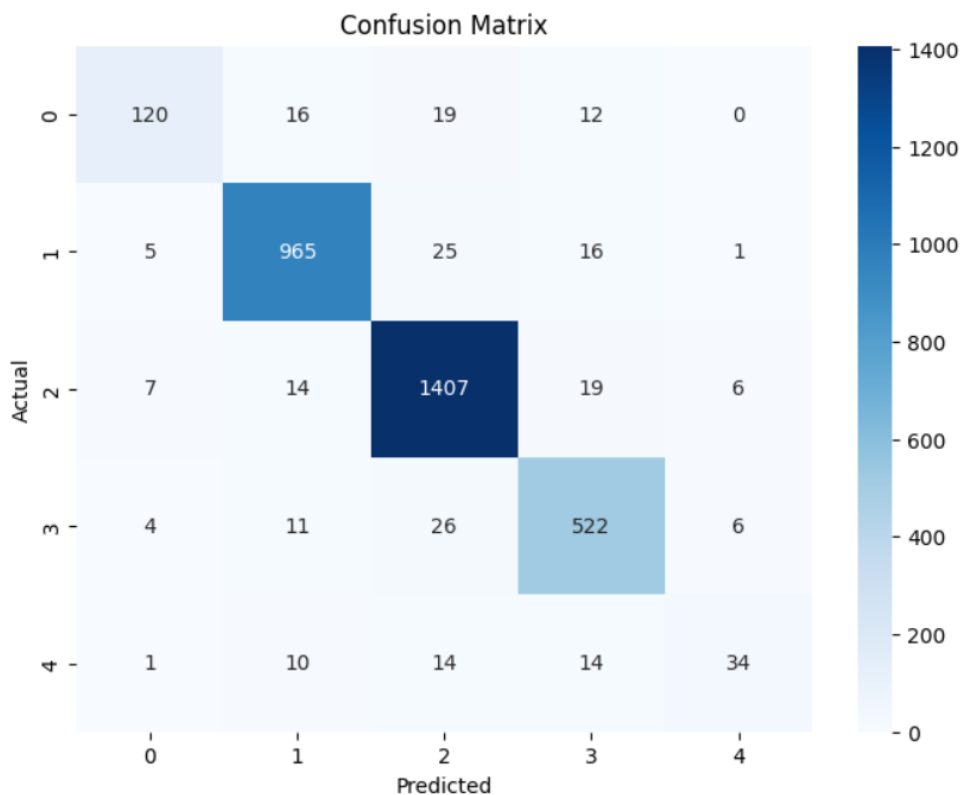


Figure 2: Confusion Matrix for Synthetic Behavior Prediction Model

The confusion matrix for the synthetic dataset shows a strong concentration of values along the diagonal, indicating that the model is able to correctly classify most behavioral outcomes. Classes such as *cautious* and *avoidant* exhibit particularly high correct prediction counts, reflecting the structured relationships defined during dataset generation.

Misclassifications are relatively minimal and are mostly observed between closely related behavioral categories such as *analytical* and *cautious*, or *impulsive* and *avoidant*. This is expected, as these behaviors share overlapping characteristics under certain contextual conditions.

Accuracy: 0.9309712889431888

Classification Report:

	precision	recall	f1-score	support
analytical	0.88	0.72	0.79	167
avoidant	0.95	0.95	0.95	1012
cautious	0.94	0.97	0.96	1453
impulsive	0.90	0.92	0.91	569
proactive	0.72	0.47	0.57	73
accuracy			0.93	3274
macro avg	0.88	0.80	0.83	3274
weighted avg	0.93	0.93	0.93	3274

Figure 3: Performance Metrics for Synthetic Model

The model achieves an overall accuracy of approximately 93%, demonstrating strong predictive capability in a controlled environment. The weighted F1-score also remains high, indicating balanced performance across most classes.

However, a noticeable variation is observed in the *proactive* class, where recall is comparatively lower. This can be attributed to the smaller number of samples for this class and its less deterministic relationship with input features. Despite this, the overall performance remains consistent and reliable.

The high accuracy of the synthetic model can be attributed to:

- Controlled feature relationships in dataset generation
- Reduced noise and ambiguity in labels
- Well-defined mappings between emotion, context, and behavior

11.2 GoEmotions Dataset Evaluation

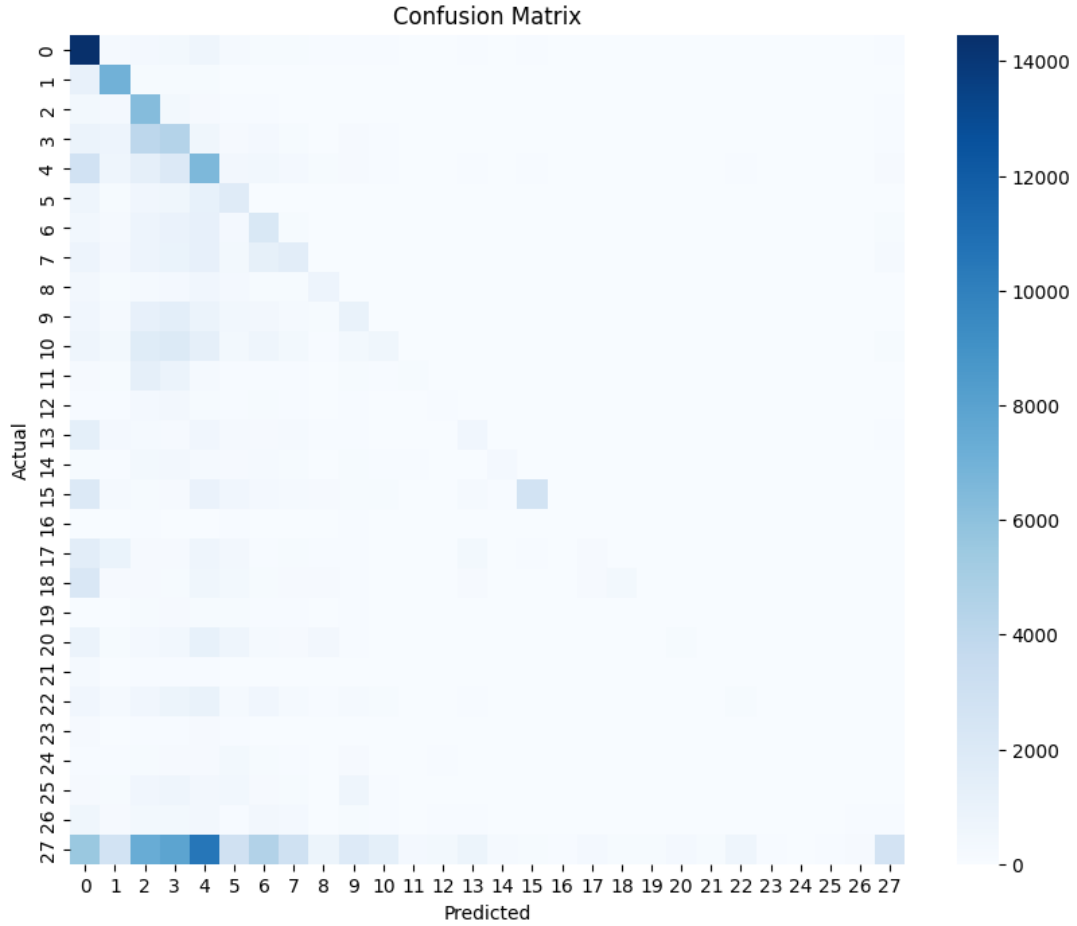


Figure 4: Confusion Matrix for GoEmotions Emotion Classification Model

The confusion matrix for the GoEmotions dataset exhibits a more dispersed pattern, with fewer strong diagonal concentrations compared to the synthetic model. This indicates that the emotion classification task is significantly more complex and less deterministic.

A high level of overlap is observed between multiple emotion classes, reflecting the inherent ambiguity in human emotional expression. Many emotions share similar linguistic patterns, leading to increased misclassification.

Accuracy: 0.03310796179110224

Classification Report:

	precision	recall	f1-score	support
0	0.37	0.84	0.51	17131
1	0.41	0.89	0.56	9245
2	0.21	0.86	0.33	8084
3	0.19	0.78	0.31	13618
4	0.20	0.72	0.31	17620
5	0.15	0.88	0.26	5999
6	0.15	0.81	0.25	7359
7	0.16	0.79	0.27	9692
8	0.15	0.89	0.26	3817
9	0.14	0.81	0.24	8469
10	0.17	0.79	0.27	11424
11	0.16	0.89	0.27	5301
12	0.11	0.88	0.19	2476
13	0.14	0.86	0.24	5629
14	0.20	0.92	0.33	3197
15	0.57	0.91	0.70	11625
16	0.10	0.89	0.18	673
17	0.20	0.86	0.33	7983
18	0.41	0.93	0.57	8191
19	0.09	0.88	0.17	1810
20	0.19	0.82	0.31	8715
21	0.08	0.85	0.14	1302
22	0.13	0.79	0.22	8785
23	0.10	0.88	0.17	1289
24	0.22	0.94	0.36	2525
25	0.20	0.88	0.33	6758
26	0.18	0.88	0.31	5514
27	0.44	0.76	0.56	55289
micro avg	0.22	0.82	0.35	249520
macro avg	0.21	0.85	0.32	249520
weighted avg	0.28	0.82	0.40	249520
samples avg	0.26	0.82	0.37	249520

Figure 5: Performance Metrics for GoEmotions Model

The overall accuracy of the GoEmotions model is relatively low (approximately 3.3%),

which is expected given the multi-label nature and high class imbalance of the dataset. Despite this, the recall values are consistently high across most classes, indicating that the model is able to identify relevant emotions but struggles with precise classification.

The discrepancy between precision and recall highlights the difficulty of emotion classification in real-world data. The model tends to predict multiple possible emotions, increasing recall but reducing precision.

Key factors affecting performance include:

- High class imbalance across emotion categories
- Overlapping emotional expressions in text
- Multi-label complexity of the dataset
- Noise and variability in real-world data

11.3 Comparative Analysis

A clear contrast is observed between the synthetic and GoEmotions models.

The synthetic model operates in a controlled environment with structured relationships, resulting in high accuracy and stable predictions. In contrast, the GoEmotions model deals with real-world data, where emotional expressions are ambiguous, overlapping, and context-dependent.

This difference highlights an important aspect of the HDT system:

- The synthetic model ensures reliable behavioral prediction under defined conditions
- The GoEmotions model captures real-world emotional variability

Together, these components enable the system to balance accuracy with realism.

11.4 Key Observations

- The system achieves high performance in behavior prediction due to structured feature integration.
- Emotion classification remains a challenging task due to the complexity of natural language.
- The combination of synthetic and real-world models improves the robustness of the overall system.

- The HDT framework successfully demonstrates the ability to simulate human-like decision patterns by integrating emotion and context.

11.5 Discussion

The results demonstrate that while high accuracy can be achieved in controlled environments, real-world scenarios introduce significant complexity. This validates the design choice of combining synthetic data with real-world emotion detection.

The HDT system does not rely solely on perfect classification but instead focuses on generating meaningful and interpretable behavioral predictions. This makes the system more practical and adaptable for real-world applications.

Overall, the evaluation confirms that the proposed approach effectively balances interpretability, realism, and predictive performance.

12 Real-Time Simulation

The Human Digital Twin (HDT) system is designed to simulate human decision-making behavior in real time by integrating emotional intelligence with contextual understanding. This section describes the interactive simulation process, user interface design, system workflow, and real-time prediction capabilities.

12.1 Overview of Real-Time Simulation

The real-time simulation module allows users to input a situation in natural language along with contextual parameters such as scenario type and difficulty level. Based on these inputs, the system processes the data through multiple stages including emotion detection, contextual integration, and behavioral prediction to generate meaningful outputs.

The simulation is implemented using a Streamlit-based interface, providing a user-friendly environment for interaction and visualization.

12.2 User Interface Design

The interface is designed to be simple, intuitive, and interactive, ensuring ease of use for users without technical expertise.

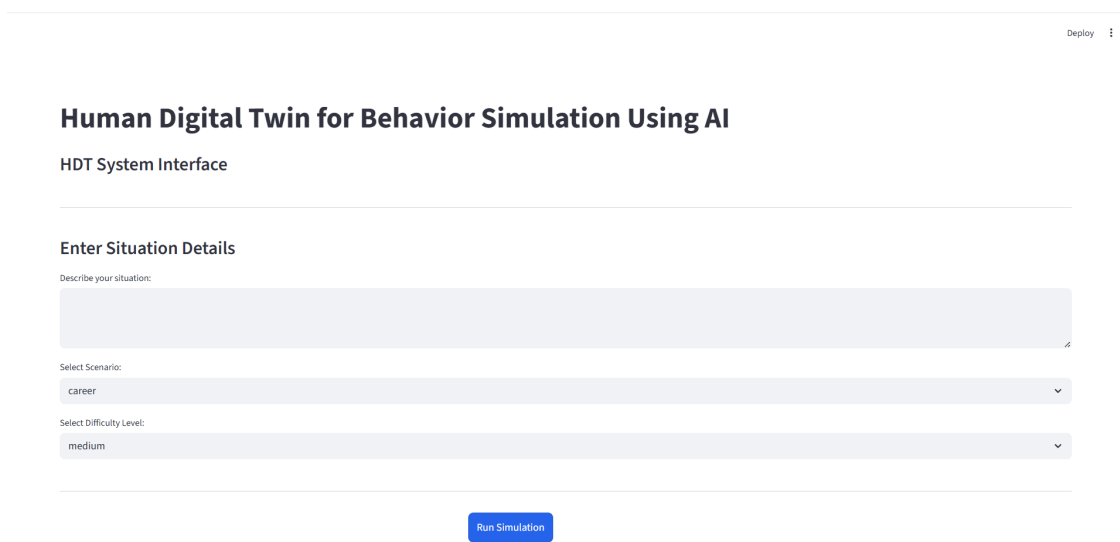
The image shows a web application interface for the Human Digital Twin (HDT) system. At the top right, there is a 'Deploy' button with a dropdown arrow. The main heading is 'Human Digital Twin for Behavior Simulation Using AI', followed by the subtitle 'HDT System Interface'. Below this, there is a section titled 'Enter Situation Details'. It contains three input fields: a large text area for 'Describe your situation', a dropdown menu for 'Select Scenario' with 'career' selected, and another dropdown menu for 'Select Difficulty Level' with 'medium' selected. At the bottom center, there is a blue button labeled 'Run Simulation'.

Figure 6: HDT System Main Interface

The main interface consists of:

- Text input area for describing the situation

- Scenario selection dropdown
- Difficulty level selection dropdown
- Simulation trigger button

12.3 Scenario Selection Mechanism

The system provides predefined scenarios representing real-world situations such as career, money, ethics, relationship, health, risk, conflict, and deadline.

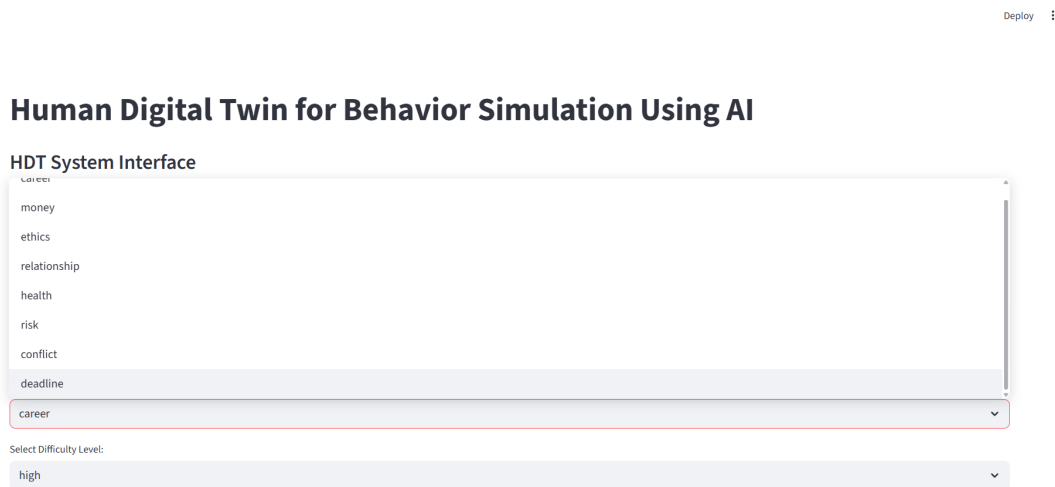


Figure 7: Scenario Selection Options

Each scenario is associated with specific difficulty levels, ensuring realistic contextual modeling.

12.4 Input Processing Example



Deploy

Human Digital Twin for Behavior Simulation Using AI

HDT System Interface

Enter Situation Details

Describe your situation:

I have a new job opportunity but I feel nervous and unsure

Select Scenario:

career

Select Difficulty Level:

high

Figure 8: User Input with Scenario and Difficulty Selection

In this example:

- User provides a textual situation
- Scenario is selected as **career**
- Difficulty level is selected as **high**

This input is passed into the simulation pipeline for processing.

12.5 Simulation Execution

Once the user clicks the "Run Simulation" button, the system executes the following steps:

- Text is preprocessed and vectorized using TF-IDF
- Emotion is detected using the trained GoEmotions model
- Detected emotion is mapped to a behavioral emotional state
- Contextual features are combined with emotional features
- Behavior prediction model generates decision output

12.6 Behavioral Simulation Output

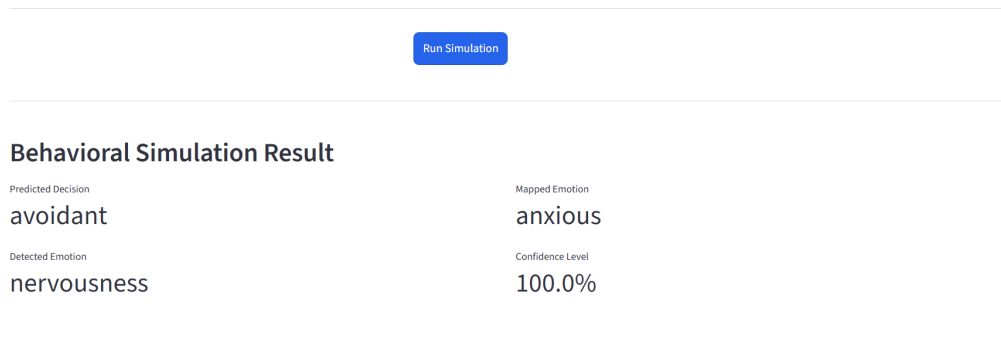


Figure 9: Behavioral Simulation Result

The system displays:

- Predicted decision (e.g., avoidant)
- Detected emotion (e.g., nervousness)
- Mapped emotional state (e.g., anxious)
- Confidence level of prediction

12.7 Emotion Confidence Visualization

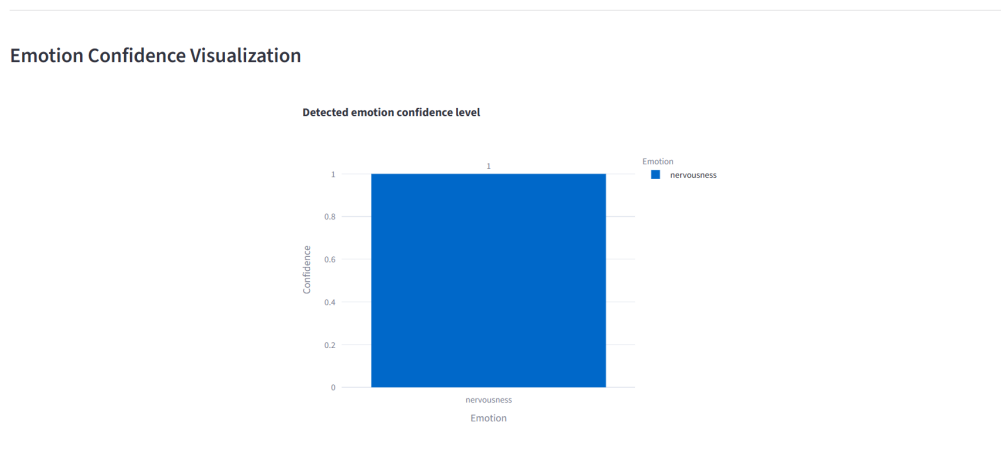


Figure 10: Emotion Confidence Visualization

A graphical representation is provided to show the confidence level of the detected emotion, enhancing interpretability.

12.8 Decision Prediction Visualization

Predicted Decision Tendency ↗



Figure 11: Predicted Decision Visualization

This visualization represents the confidence of the predicted behavioral decision, helping users understand prediction reliability.

12.9 Emotion to Behavior Mapping

Emotion to Behavior Mapping

Detected Emotion: nervousness Mapped Behavioral State: anxious Predicted Decision Tendency: avoidant

Explanation

The dominant detected emotion is 'nervousness', which maps to the behavioral state 'anxious'. For the selected scenario 'career' with difficulty level 'high', the predicted decision tendency is 'avoidant'.

Behavioral Interpretation

Feeling anxious in challenging situations can increase the tendency to avoid decisions. Higher difficulty levels can increase emotional pressure and effect decision patterns. Using structured planning and breaking the problem into smaller steps may help reduce avoidance.

Figure 12: Emotion to Behavior Mapping and Interpretation

The system clearly maps:

- Detected emotion
- Behavioral emotional state
- Final decision tendency

12.10 Explanation Module

The explanation module provides a transparent description of how the system arrived at the prediction. It connects emotion detection with contextual parameters to justify the decision outcome.

This improves trust and interpretability of the system.

12.11 Behavioral Interpretation Module

The behavioral interpretation module offers human-understandable insights into the predicted decision. It explains how emotional state and difficulty level influence behavior and suggests possible improvements.

For example:

- High stress situations increase avoidance tendencies
- Structured planning can improve decision-making

12.12 End-to-End Simulation Flow

The complete real-time simulation follows this pipeline:

- User Input (Text + Scenario + Difficulty)
- Emotion Detection (GoEmotions Model)
- Emotion Mapping
- Context Integration
- Feature Encoding and Scaling
- Behavior Prediction Model
- Explanation Generation
- Behavioral Interpretation
- Final Output Display

12.13 System Characteristics

The real-time simulation system demonstrates the following key characteristics:

- Interactive and user-friendly interface
- Real-time prediction capability
- Integration of emotion and context
- Explainable AI approach
- Scalable architecture

12.14 Summary

The real-time simulation module effectively demonstrates the practical implementation of the Human Digital Twin framework. By combining emotional intelligence, contextual reasoning, and machine learning models, the system provides realistic and interpretable behavioral predictions in real time.

13 Advantages

The proposed Human Digital Twin (HDT) system introduces several significant advantages over traditional behavior modeling approaches by combining emotional intelligence, contextual reasoning, and interpretable artificial intelligence.

13.1 Integration of Emotion and Context

One of the major strengths of the system is its ability to integrate emotional states with contextual parameters. Unlike conventional systems that rely solely on either textual sentiment or contextual rules, this model combines both dimensions to simulate realistic human behavior.

This dual integration allows the system to better capture the complexity of human decision-making, where emotions and situational factors jointly influence outcomes.

13.2 Realistic Behavioral Simulation

The system generates behavior predictions that closely resemble real human tendencies. By mapping emotions to behavioral states and incorporating difficulty levels, the model simulates how individuals react under different pressures and scenarios.

This makes the system suitable for applications requiring realistic human-like responses.

13.3 Use of Synthetic Dataset for Controlled Learning

Due to the lack of publicly available datasets combining emotion, context, and behavior, a synthetic dataset is constructed. This provides several advantages:

- Controlled data generation with well-defined relationships
- Balanced representation of decision classes
- Flexibility to model different behavioral conditions

This controlled setup improves the reliability and consistency of the behavior prediction model.

13.4 High Accuracy in Behavioral Prediction

The behavior prediction model trained on the synthetic dataset achieves high accuracy and strong classification performance. The confusion matrix and evaluation metrics demonstrate that the system can effectively distinguish between different decision tendencies.

This indicates that the model is capable of capturing meaningful patterns in behavioral data.

13.5 Multi-Label Emotion Detection Capability

The system leverages the GoEmotions dataset to perform multi-label emotion classification. This allows detection of complex emotional expressions rather than limiting predictions to a single emotion.

Such capability improves the realism of emotion detection and enhances downstream behavioral prediction.

13.6 Interpretability and Explanation Module

A key advantage of the system is the inclusion of an explanation module. Instead of providing only predictions, the system explains:

- Detected emotion
- Mapped behavioral state
- Reasoning behind the decision

Although advanced explainable AI techniques such as SHAP or LIME are not used, the system provides structured and human-readable explanations by linking emotional detection with contextual parameters and predicted outcomes. This improves transparency and user trust.

13.7 Behavioral Interpretation Layer

Beyond explanation, the system includes a behavioral interpretation module that provides human-understandable insights. It helps users understand how emotional and contextual factors influence decisions and suggests possible improvements.

This makes the system not just predictive, but also advisory.

13.8 Real-Time Interactive Simulation

The Streamlit-based interface enables real-time interaction with the system. Users can input their situations and instantly receive predictions, visualizations, and interpretations.

This enhances usability and demonstrates the practical applicability of the model.

13.9 Scalable and Modular Architecture

The system is designed using a modular architecture, where each component (emotion detection, feature engineering, prediction, and interface) operates independently.

This allows:

- Easy updates and improvements
- Integration of advanced models in future
- Deployment in different environments

13.10 Flexibility Across Multiple Domains

The inclusion of multiple scenarios such as career, health, finance, and relationships makes the system adaptable across different real-world domains.

This increases its applicability in diverse fields such as:

- Decision support systems
- Human-computer interaction
- Psychological modeling

13.11 Improved Decision Understanding

The system not only predicts decisions but also helps users understand the underlying factors influencing those decisions. This provides deeper insight into human behavior.

13.12 Summary

Overall, the proposed HDT system provides a powerful combination of emotion-aware intelligence, contextual reasoning, real-time simulation, and interpretability, making it a robust and advanced solution for human behavior modeling.

14 Limitations

While the proposed Human Digital Twin (HDT) system demonstrates strong capabilities in behavior simulation, certain limitations exist due to design constraints, data availability, and modeling assumptions. These limitations provide important insights into the current scope of the system and highlight opportunities for future improvements.

14.1 Dependence on Synthetic Behavioral Dataset

A significant limitation of the system is its reliance on a synthetically generated dataset for behavior prediction. Although the dataset is carefully designed using logical mappings between emotion, context, and decision outcomes, it does not fully capture the randomness and unpredictability of real human behavior.

As a result, the model performs effectively in structured environments but may not generalize perfectly to real-world scenarios.

14.2 Limited Availability of Real-World Behavioral Data

Currently, there is no publicly available dataset that combines emotional states, contextual conditions, and behavioral decisions in a unified format. Due to this limitation, the system cannot be trained on real-world behavioral data, restricting its ability to learn naturally occurring human decision patterns.

14.3 Imbalance in GoEmotions Dataset

The GoEmotions dataset used for emotion classification contains significant class imbalance across emotion categories. Some emotions are highly represented, while others have limited samples.

This imbalance affects the model’s ability to learn all classes equally, resulting in lower precision and overall classification accuracy.

14.4 Lower Accuracy in Emotion Classification

Compared to the behavior prediction model, the emotion classification model exhibits lower accuracy. This is due to the inherent complexity of multi-label classification and the ambiguity present in natural language.

Errors in emotion detection can propagate to the behavior prediction stage, affecting overall system performance.

14.5 Limited Emotional Coverage

The system reduces a wide range of emotions into a smaller set of behavioral emotional states such as anxious, stressed, and thinking. While this simplification improves integration and stability, it limits the system’s ability to capture fine-grained emotional nuances.

Subtle emotional differences and mixed emotional states are not fully represented, which may impact the depth of behavioral simulation.

14.6 Limited Behavioral Mapping

The system maps emotional and contextual inputs to a predefined set of behavioral outcomes such as analytical, cautious, avoidant, impulsive, and proactive. While this structured mapping enables effective classification, it does not fully represent the wide spectrum of possible human behaviors.

Real-world decisions often involve complex, dynamic, and mixed behavioral patterns that are not captured in the current model.

14.7 Simplified Decision Modeling

The decision-making process in the system is based on predefined logical rules and learned patterns from synthetic data. While effective for modeling general behavior, this approach simplifies the complexity of human decision-making.

Factors such as emotional intensity variation, past experiences, and long-term consequences are not fully incorporated into the model.

14.8 Limited Contextual Understanding

The system considers a fixed set of contextual parameters such as scenario type, difficulty level, stress level, personality, and age group. However, real-world decision-making is influenced by a much broader range of factors including social dynamics, cultural background, environment, and psychological conditions.

The absence of these factors limits the depth and realism of context-aware behavior simulation.

14.9 Static Scenario Categories

The system operates on a predefined set of scenarios such as career, money, ethics, relationship, health, risk, conflict, and deadline. While these categories cover common situations,

they are static and do not dynamically adapt to new or complex real-world scenarios.

This limits the flexibility of the system in handling diverse or evolving contexts.

14.10 Absence of Advanced Explainability Techniques

Although the system includes an explanation module, it does not utilize advanced explainable AI techniques such as SHAP or LIME for feature-level interpretability.

The current explanation mechanism is rule-based and descriptive, which improves user understanding but does not provide deep insights into model decision boundaries.

14.11 Static Model Behavior

The system uses pre-trained models that do not update based on new user interactions. It lacks adaptive learning capabilities, meaning it cannot improve or personalize its predictions over time.

This limits its ability to evolve with changing behavioral patterns.

14.12 Limited Personalization

The system does not maintain individual user profiles or historical behavior patterns. Predictions are generated based on generalized inputs rather than personalized data.

As a result, the system cannot fully adapt to individual differences in decision-making behavior.

14.13 Scalability to Complex Real-World Scenarios

While the system performs well for predefined structured scenarios, it may struggle with highly complex, ambiguous, or multi-layered real-world situations where multiple contexts interact simultaneously.

14.14 Summary

Despite these limitations, the proposed HDT system provides a strong foundation for integrating emotion and context in behavior simulation. The identified constraints highlight important directions for future improvements, including real-world data integration, advanced modeling techniques, dynamic adaptation, and enhanced interpretability.

15 Future Work

The proposed Human Digital Twin (HDT) system provides a strong foundation for behavior simulation by integrating emotional intelligence and contextual reasoning. However, there are several opportunities to enhance the system further by addressing current limitations and extending its capabilities. The following directions outline potential future improvements.

15.1 Expansion of Emotion–Behavior Mapping

The current system maps a wide range of detected emotions into a limited set of behavioral emotional states. Future work can focus on expanding this mapping to include more fine-grained emotional categories and corresponding behavioral responses.

This would enable the system to capture subtle emotional variations and produce more precise and realistic behavioral predictions.

15.2 Incorporation of Real-World Behavioral Data

One of the most important future improvements is the integration of real-world behavioral datasets. Instead of relying primarily on synthetic data, future systems can utilize real human decision-making data collected from surveys, experiments, or real-life interactions.

This would improve the model’s ability to generalize and reflect authentic human behavior patterns.

15.3 Multi-Turn Context Analysis

The current system processes each input independently without considering previous interactions. Future work can introduce multi-turn context analysis, where the system maintains a history of user inputs and decisions.

This would allow the model to simulate behavior over time and capture how decisions evolve across multiple situations, making the system more dynamic and realistic.

15.4 Adaptive Behavioral Learning

The system currently uses static pre-trained models that do not update based on user interactions. Future enhancements can include adaptive learning mechanisms, where the model continuously learns from new inputs and feedback.

This would enable the system to improve its predictions over time and adapt to changing behavioral patterns.

15.5 Personalized Digital Twin Modeling

A significant advancement would be the development of personalized Human Digital Twins. Instead of using generalized profiles, the system can maintain individual user profiles, including behavioral history, preferences, and emotional patterns.

This would allow the system to generate highly personalized predictions tailored to specific individuals.

15.6 Integration of Advanced Explainability Techniques

Future versions of the system can incorporate advanced explainable AI techniques such as SHAP or LIME to provide deeper insights into model predictions. These techniques can help identify feature-level contributions and improve transparency.

This would strengthen the interpretability of the system and make it more suitable for critical applications.

15.7 Enhanced Contextual Modeling

The current system considers a limited set of contextual factors. Future work can expand this by incorporating additional variables such as social environment, cultural background, time pressure, and external influences.

This would result in a more comprehensive and realistic representation of human decision-making.

15.8 Dynamic Scenario Generation

Instead of relying on predefined static scenarios, future systems can generate dynamic scenarios based on user input or real-world data. This would improve flexibility and allow the system to handle a wider variety of situations.

15.9 Integration with Deep Learning Models

The current implementation primarily uses classical machine learning techniques. Future work can explore deep learning approaches such as transformer-based models for emotion detection and sequence modeling.

These models can improve performance, especially for complex language understanding tasks.

15.10 Real-Time Continuous Simulation

Future systems can extend the current simulation into continuous real-time behavior tracking, where decisions are simulated dynamically over time rather than as isolated predictions.

This would enhance the realism of the Human Digital Twin framework.

15.11 Cross-Domain Applications

The HDT system can be extended to various domains such as healthcare, education, finance, and organizational behavior. Domain-specific adaptations can improve its effectiveness in specialized applications.

15.12 Deployment and Scalability Enhancements

Future work can focus on deploying the system in cloud environments with scalable architectures. This would allow handling multiple users simultaneously and improve system performance in real-world applications.

15.13 Summary

The future enhancements outlined above aim to transform the current HDT system into a more advanced, adaptive, and realistic human behavior simulation framework. By integrating real-world data, personalization, adaptive learning, and advanced modeling techniques, the system can evolve into a powerful tool for understanding and predicting human decision-making at a deeper level.

16 Conclusion

The Human Digital Twin (HDT) system presented in this work provides a comprehensive approach to simulating human behavior by integrating emotional intelligence with contextual reasoning. The primary objective of the system was to model how individuals make decisions under varying emotional and situational conditions, and the results demonstrate that this objective has been successfully achieved within the defined scope.

The system effectively combines multiple components, including text preprocessing, feature extraction using TF-IDF, emotion classification using the GoEmotions dataset, synthetic behavioral data modeling, and a behavior prediction model. Each of these components contributes to the overall functionality of the system, forming a complete pipeline that transforms raw textual input into meaningful behavioral predictions.

One of the key strengths of the system lies in its ability to integrate emotional states with contextual parameters such as scenario type and difficulty level. This integration enables the system to simulate realistic decision-making patterns, reflecting how internal emotions and external conditions interact in real-world situations.

The use of a synthetic dataset for behavior prediction plays a crucial role in enabling structured learning of decision patterns. While real-world behavioral datasets are limited, the synthetic dataset provides a controlled environment where relationships between variables can be clearly defined. This results in high accuracy and consistent performance in behavior prediction.

At the same time, the use of the GoEmotions dataset introduces real-world complexity into the system. The lower accuracy observed in emotion classification highlights the inherent challenges of understanding human emotions from textual data. This contrast between controlled and real-world performance provides valuable insights into the strengths and limitations of current approaches.

Another important contribution of the system is the inclusion of interpretability features. The explanation module and behavioral interpretation layer ensure that the system does not operate as a black box. Instead, it provides clear reasoning behind predictions, allowing users to understand how decisions are influenced by emotional and contextual factors.

The development of a real-time simulation interface further enhances the practical applicability of the system. Through the Streamlit-based interface, users can interact with the system, provide inputs, and observe predictions along with visualizations and explanations. This demonstrates the feasibility of deploying the HDT framework in real-world applications.

Despite its strengths, the system also acknowledges several limitations, including dependence on synthetic data, simplified emotional representation, limited contextual factors, and

lack of adaptive learning. These limitations provide direction for future improvements and highlight the need for more advanced datasets and modeling techniques.

The future work outlined in this report presents a clear path for enhancing the system, including the integration of real-world behavioral data, expansion of emotion modeling, personalization, adaptive learning, and advanced explainability techniques. These improvements have the potential to transform the current system into a more advanced and realistic human behavior simulation framework.

In conclusion, the proposed Human Digital Twin system successfully demonstrates the feasibility of combining emotion detection, contextual modeling, and machine learning to simulate human decision-making behavior. The system not only provides accurate predictions in controlled environments but also highlights the challenges of real-world emotion understanding.

The HDT framework represents an important step toward developing intelligent systems that can understand, simulate, and interpret human behavior. With further advancements and refinements, such systems can play a significant role in areas such as decision support, human-computer interaction, psychological analysis, and intelligent simulation environments.

References

- [1] M. Grieves and J. Vickers, *Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behavior in Complex Systems*. Cham, Switzerland: Springer, 2017.
- [2] F. Tao, M. Zhang, Y. Liu, and A. Y. C. Nee, “Digital twin in industry: State-of-the-art,” *IEEE Transactions on Industrial Informatics*, vol. 15, no. 4, pp. 2405–2415, 2019.
- [3] D. Demszky, D. Movshovitz-Attias, J. Ko *et al.*, “GoEmotions: A dataset of fine-grained emotions,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 4040–4054, 2020.
- [4] B. Shneiderman, “Human-centered artificial intelligence: Reliable, safe & trustworthy,” *Communications of the ACM*, vol. 63, no. 1, pp. 56–65, 2020.

Appendix

This appendix contains the complete implementation of the Human Digital Twin (HDT) system, including dataset generation, preprocessing, model training, simulation pipeline, and user interface code.

A. Synthetic Dataset Generation Code

```
import pandas as pd
import random
import numpy as np
random.seed(42)
np.random.seed(42)
import joblib
print("Libraries loaded successfully")
scenarios = ["career", "money", "ethics", "relationship", "health", "risk", "
    conflict", "deadline"]
difficulty_map = {"career" : ["medium", "high", "very high"], "money" : ["medium
    ", "high"], "ethics" : ["high", "very high"], "relationship" : ["medium", "
    high"], "health" : ["high", "very high"], "risk" : ["low", "medium", "high"],
    "conflict" : ["medium", "high"], "deadline" : ["high", "very high"]}
emotion_map = {"low" : ["calm", "focused"], "medium" : ["concerned", "thinking"],
    "high" : ["anxious", "stressed"], "very high" : ["panic", "overwhelmed"]}
print("Categories defined successfully")
def generate_behavior_row():
    scenario = random.choice(scenarios)
    difficulty = random.choice(difficulty_map[scenario])
    emotional_state = random.choice(emotion_map[difficulty])
    personality = random.choice(["introvert", "extrovert", "balanced"])
    age_group = random.choice(["young", "middle", "senior"])
    stress_level = random.choice(["low", "medium", "high"])
    decision_context = random.choice(["personal", "professional", "social"])
    if emotional_state in ["calm", "focused"]:
        base_decision = "analytical"
    elif emotional_state in ["concerned", "thinking"]:
        base_decision = "cautious"
    elif emotional_state in ["anxious", "stressed"]:
        base_decision = "avoidant"
    elif emotional_state in ["panic", "overwhelmed"]:
```

```

        base_decision = "impulsive"
    else:
        base_decision = "proactive"
    if stress_level == "high" and base_decision == "analytical":
        base_decision = "cautious"
    if stress_level == "low" and base_decision == "avoidant":
        base_decision = "cautious"
    if random.random() < 0.08:
        base_decision = random.choice(["analytical", "avoidant", "cautious", "
impulsive", "proactive"])
    decision = base_decision
    response_time_map = {"low" : ["fast", "moderate"], "medium" : ["moderate", "
slow"], "high" : ["slow", "very slow"], "very high" : ["very slow"]}
    response_time = random.choice(response_time_map[difficulty])
    return [scenario, difficulty, emotional_state, decision, response_time,
    personality, age_group, stress_level, decision_context]
data = [generate_behavior_row() for i in range(5000)]
df = pd.DataFrame(data, columns = ["scenario", "difficulty", "emotion", "decision
", "response_time", "personality", "age_group", "stress_level", "context"])
df = df.drop_duplicates()
print("Rows generated", len(df))
print(df.shape)
display(df.head())
joblib.dump(scenarios, "../models/scenarios.pkl")
joblib.dump(difficulty_map, "../models/difficulty_map.pkl")
print("Scenarios and difficulty map saved successfully")
df.to_csv("../data/synthetic.csv", index=False)
print("Synthetic dataset created successfully")

```

B. Preprocessing and Feature Encoding

```

import pandas as pd
from sklearn.preprocessing import OneHotEncoder
import numpy as np
import joblib
synthetic_path = ("../data/synthetic.csv")
df = pd.read_csv(synthetic_path)
print("\n=== Dataset Shape ===")

```

```

print(df.shape)
print("\n=== Dataset Info ===")
display(df.head())
features = df.drop(columns = ["decision"])
target = df["decision"]
print("Features shape")
print(features.shape)
print("Target shape")
print(target.shape)
encoder = OneHotEncoder(sparse_output = False)
features_encoded = encoder.fit_transform(features)
print("Encoded feature shape")
print(features_encoded.shape)
np.save("../data/synthetic_features.npy", features_encoded)
np.save("../data/synthetic_labels.npy", target)
joblib.dump(encoder, "../models/synthetic_encoder.pkl")
print("All Saved Successfully")

```

C. Feature Engineering and Scaling

```

import numpy as np
from sklearn.preprocessing import StandardScaler
import joblib
features_encoded = np.load("../data/synthetic_features.npy")
target = np.load("../data/synthetic_labels.npy", allow_pickle = True)
print("Loaded features")
print(features_encoded.shape)
print("Loaded target")
print(target.shape)
scaler = StandardScaler()
features_scaled = scaler.fit_transform(features_encoded)
print("Scaled features shape")
print(features_scaled.shape)
np.save("../data/synthetic_features_scaled.npy", features_scaled)
np.save("../data/synthetic_labels_final.npy", target)
joblib.dump(scaler, "../models/synthetic_scaler.pkl")
print("Feature engineering complete Scaled features, final labels and scaler
      saved.")

```

D. Behavior Prediction Model Training

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import accuracy_score, classification_report,
    confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
import joblib
synthetic_features = np.load("../data/synthetic_features_scaled.npy")
synthetic_labels = np.load("../data/synthetic_labels_final.npy", allow_pickle =
    True)
print("Feature Shape")
print(synthetic_features.shape)
print("Label Shape")
print(synthetic_labels.shape)
features_train, features_test, labels_train, labels_test = train_test_split(
    synthetic_features, synthetic_labels, test_size = 0.2, random_state = 42,
    stratify = synthetic_labels)
print("Training Feature Shape")
print(features_train.shape)
print("Testing Feature Shape")
print(features_test.shape)
behavior_prediction_model = HistGradientBoostingClassifier(max_iter = 300,
    learning_rate = 0.07, max_depth = 10, random_state = 42)
behavior_prediction_model.fit(features_train, labels_train)
print("Behavior Prediction Model Trained Successfully")
predicted_labels = behavior_prediction_model.predict(features_test)
model_accuracy = accuracy_score(labels_test, predicted_labels)
print("Model Accuracy")
print(model_accuracy)
print(classification_report(labels_test, predicted_labels))
conf_matrix = confusion_matrix(labels_test, predicted_labels)
plt.figure(figsize = (8, 6))
sns.heatmap(conf_matrix, annot = True, fmt = 'd', cmap = 'Blues')
plt.title("Confusion Matrix - Synthetic Behavior Model")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
```



```
plt.show()
joblib.dump(behavior_prediction_model, "../models/synthetic_model.pkl")
print("Model Saved Successfully")
```

E. GoEmotions Data Processing

```
import pandas as pd
import numpy as np
import re
import joblib

goemotions_raw = pd.read_csv("../data/goemotions.csv")
print("Original Dataset Shape")
print(goemotions_raw.shape)
display(goemotions_raw.head())
emotion_label_columns = goemotions_raw.columns[9:]
goemotions_filtered = goemotions_raw[["text"] + list(emotion_label_columns)]
print("Filtered Dataset Shape")
print(goemotions_filtered.shape)
goemotions_filtered = goemotions_filtered[goemotions_filtered[
    emotion_label_columns].sum(axis = 1) > 0]
print("After Removing Empty Emotion Rows")
print(goemotions_filtered.shape)
def clean_text_function(text):
    text = str(text).lower()
    text = re.sub(r"http\S+", " ", text)
    text = re.sub(r"[^a-zA-Z\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text
goemotions_filtered["clean_text"] = goemotions_filtered["text"].apply(
    clean_text_function)
goemotions_filtered = goemotions_filtered[goemotions_filtered["clean_text"].str.
    strip() != ""]
print("After Cleaning Text")
print(goemotions_filtered.shape)
display(goemotions_filtered[["text", "clean_text"]].head())
goemotions_processed = goemotions_filtered.drop(columns = ["text"])
print("Final Preprocessed Dataset Shape")
print(goemotions_processed.shape)
```

```

display(goemotions_processed.head())
joblib.dump(list(emotion_label_columns), "../models/goemotions_label_names.pkl")
goemotions_processed.to_csv("../data/goemotions_processed.csv", index = False)
print("Emotion label names saved successfully")
print("GoEmotions preprocessing completed successfully")

```

F. GoEmotions Feature Engineering

```

import pandas as pd
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
import joblib
goemotions_processed = pd.read_csv("../data/goemotions_processed.csv")
print("Dataset Shape")
print(goemotions_processed.shape)
display(goemotions_processed.head())
texts = goemotions_processed["clean_text"]
emotion_columns = goemotions_processed.columns.drop("clean_text")
labels = goemotions_processed[emotion_columns]
print("Texts Shape")
print(texts.shape)
print("Labels Shape")
print(labels.shape)
tfidf_vectorizer = TfidfVectorizer(max_features = 10000, ngram_range = (1, 2),
    min_df = 3, max_df = 0.95, stop_words = 'english')
text_features = tfidf_vectorizer.fit_transform(texts)
print("TF-IDF Feature Shape")
print(text_features.shape)
joblib.dump(text_features, "../data/goemotions_tfidf_features.pkl")
np.save("../data/goemotions_labels.npy", labels.values)
joblib.dump(tfidf_vectorizer, "../models/goemotions_vectorizer.pkl")
print("GoEmotions Feature Engineering Completed Successfully")

```

G. Emotion Classification Model Training

```

import numpy as np
import joblib

```

```

from sklearn.model_selection import train_test_split
from sklearn.multiclass import OneVsRestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score, classification_report, hamming_loss
text_features = joblib.load("../data/goemotions_tfidf_features.pkl")
emotion_labels = np.load("../data/goemotions_labels.npy")
print("Features Shape")
print(text_features.shape)
print("Labels Shape")
print(emotion_labels.shape)
X_train, X_test, y_train, y_test = train_test_split(text_features, emotion_labels
    , test_size = 0.2, random_state = 42)
print("Train Shape")
print(X_train.shape)
print("Test Shape")
print(X_test.shape)
base_classifier = LogisticRegression(solver = 'liblinear', class_weight = '
    balanced', max_iter = 2000)
multilabel_model = OneVsRestClassifier(base_classifier)
print("Model Initialized Successfully")
multilabel_model.fit(X_train, y_train)
print("GoEmotions Multi-Label Model Trained Successfully")
y_prob = multilabel_model.predict_proba(X_test)
threshold = 0.65
y_pred = (y_prob >= threshold).astype(int)
micro_f1 = f1_score(y_test, y_pred, average = 'micro')
macro_f1 = f1_score(y_test, y_pred, average = 'macro')
hamming = hamming_loss(y_test, y_pred)
print("Micro F1 Score")
print(micro_f1)
print("Macro F1 Score")
print(macro_f1)
print("Hamming Loss")
print(hamming)
joblib.dump(multilabel_model, "../models/goemotions_model.pkl")
print("Model Saved Successfully")

```

H. Simulation Pipeline Demonstration

```
import numpy as np
import pandas as pd
import joblib

synthetic_model = joblib.load("../models/synthetic_model.pkl")
synthetic_encoder = joblib.load("../models/synthetic_encoder.pkl")
synthetic_scaler = joblib.load("../models/synthetic_scaler.pkl")
scenarios = joblib.load("../models/scenarios.pkl")
difficulty_map = joblib.load("../models/difficulty_map.pkl")
emotion_names = joblib.load("../models/goemotions_label_names.pkl")
goemotion_model = joblib.load("../models/goemotions_model.pkl")
vectorizer = joblib.load("../models/goemotions_vectorizer.pkl")
print("All model loaded successfully")

emotion_mapping = {"nervousness" : "anxious", "fear" : "anxious", "sadness" : "stressed", "anger" : "stressed", "disappointment" : "stressed", "joy" : "thinking", "optimism" : "thinking", "curiosity" : "thinking", "confusion" : "thinking", "neutral" : "thinking"}

def simulate_behavior(user_text, scenario, difficulty):
    text_vector = vectorizer.transform([user_text])
    emotion_probs = goemotion_model.predict_proba(text_vector)[0]
    dominant_index = np.argmax(emotion_probs)
    dominant_emotion = (emotion_names[dominant_index])
    mapped_emotion = emotion_mapping.get(dominant_emotion, "thinking")
    synthetic_input = pd.DataFrame([{"scenario" : scenario, "difficulty" : difficulty, "emotion" : mapped_emotion, "response_time" : "slow" if difficulty in ["high", "very high"] else "moderate", "personality" : "balanced", "age_group" : "young", "stress_level" : "medium", "context" : "professional"}])
    encoded = synthetic_encoder.transform(synthetic_input)
    scaled = synthetic_scaler.transform(encoded)
    decision = synthetic_model.predict(scaled)[0]
    probabilities = synthetic_model.predict_proba(scaled)[0]
    confidence = np.max(probabilities)
    explanation = (f"The dominant detected emotion is '{dominant_emotion}', " f"which maps to the behavioral state '{mapped_emotion}'.\n" f"For the selected scenario '{scenario}' with difficulty level '{difficulty}', " f"the predicted decision tendency is '{decision}'.")
    return {"Predicted Decision" : decision, "Confidence" : round(float(confidence), 3), "Detected Emotion" : dominant_emotion, "Mapped Emotion" :
```

```

mapped_emotion, "Explanation" : explanation}
def behavioral_interpretation(mapped_emotion, difficulty, decision):
    interpretation = []
    if mapped_emotion in ["anxious", "stressed", "panic", "overwhelmed"]:
        interpretation.append("Feeling anxious in challenging situations can
increase the tendency to avoid decisions.")
    elif mapped_emotion in ["calm", "focused"]:
        interpretation.append("A calm emotional state usually supports clear and
balanced decision-making.")
    elif mapped_emotion in ["concerned", "thinking"]:
        interpretation.append("A reflective emotional state may lead to careful
and cautious decisions.")
    else:
        interpretation.append(f"The emotional state '{mapped_emotion}' influences
how decisions are made under situational demands.")
    if difficulty in ["high", "very high"]:
        interpretation.append("Higher difficulty levels can increase emotional
pressure and affect decision patterns.")
    elif difficulty == "medium":
        interpretation.append("Moderate difficulty allows better control over
emotional reactions.")
    else:
        interpretation.append("Lower difficulty situations reduce emotional
pressure on decisions.")
    if decision == "avoidant":
        interpretation.append("Using structured planning and breaking the problem
into smaller steps may help reduce avoidance.")
    elif decision == "impulsive":
        interpretation.append("Taking a short pause before deciding may improve
decision quality.")
    elif decision == "analytical":
        interpretation.append("Continuing structured analysis can support
consistent decision outcomes.")
    elif decision == "cautious":
        interpretation.append("Balancing caution with timely action may improve
results.")
    else:

```

```

        interpretation.append("Improving emotional awareness may enhance decision
        effectiveness.")
    return " ".join(interpretation)
user_text = input("Enter your situation: ")
print("\nAvailable Scenarios:")
print(scenarios)
scenario = input("Enter scenario from above list: ").strip().lower()
if scenario not in scenarios:
    print("Invalid scenario selected. Please choose a valid option from the list
    above.")
else:
    print(f"\nAvailable Difficulties for '{scenario}':")
    print(difficulty_map[scenario])
    difficulty = input("Enter difficulty from above list: ").strip().lower()
    if difficulty not in difficulty_map[scenario]:
        print("Invalid difficulty selected. Please choose a valid option for the
        selected scenario.")
    else:
        result = simulate_behavior(user_text = user_text, scenario = scenario,
        difficulty = difficulty)
        print("\n--- Behavioral Simulation Result ---\n")
        print(f"Predicted Decision : {result['Predicted Decision']}")
        print(f"Confidence Level : {result['Confidence']}")
        print(f"Detected Emotion : {result['Detected Emotion']}")
        print(f"Mapped Emotion : {result['Mapped Emotion']}")
        print("\nExplanation:")
        print(result['Explanation'])
        print("\n--- Behavioral Interpretation ---\n")
        interpretation_text = behavioral_interpretation(mapped_emotion = result['
        Mapped Emotion'], difficulty = difficulty, decision = result['Predicted
        Decision'])
        print(interpretation_text)

```

I. Simulation Engine Implementation

```

import os
import numpy as np
import pandas as pd

```

```

import joblib
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
MODELS_DIR = os.path.join(BASE_DIR, "models")
synthetic_model = joblib.load(os.path.join(MODELS_DIR, "synthetic_model.pkl"))
synthetic_encoder = joblib.load(os.path.join(MODELS_DIR, "synthetic_encoder.pkl")
    )
synthetic_scaler = joblib.load(os.path.join(MODELS_DIR, "synthetic_scaler.pkl"))
scenarios = joblib.load(os.path.join(MODELS_DIR, "scenarios.pkl"))
difficulty_map = joblib.load(os.path.join(MODELS_DIR, "difficulty_map.pkl"))
emotion_names = joblib.load(os.path.join(MODELS_DIR, "goemotions_label_names.pkl"
    ))
goemotion_model = joblib.load(os.path.join(MODELS_DIR, "goemotions_model.pkl"))
vectorizer = joblib.load(os.path.join(MODELS_DIR, "goemotions_vectorizer.pkl"))
emotion_mapping = {"nervousness" : "anxious", "fear" : "anxious", "sadness" : "stressed", "anger" : "stressed", "disappointment" : "stressed", "joy" : "thinking", "optimism" : "thinking", "curiosity" : "thinking", "confusion" : "thinking", "neutral" : "thinking"}
def simulate_behavior(user_text, scenario, difficulty):
    text_vector = vectorizer.transform([user_text])
    emotion_probs = goemotion_model.predict_proba(text_vector)[0]
    dominant_index = np.argmax(emotion_probs)
    dominant_emotion = (emotion_names[dominant_index])
    mapped_emotion = emotion_mapping.get(dominant_emotion, "thinking")
    synthetic_input = pd.DataFrame([{"scenario" : scenario, "difficulty" : difficulty, "emotion" : mapped_emotion, "response_time" : "slow" if difficulty in ["high", "very high"] else "moderate", "personality" : "balanced", "age_group" : "young", "stress_level" : "medium", "context" : "professional"}])
    encoded = synthetic_encoder.transform(synthetic_input)
    scaled = synthetic_scaler.transform(encoded)
    decision = synthetic_model.predict(scaled)[0]
    probabilities = synthetic_model.predict_proba(scaled)[0]
    confidence = np.max(probabilities)
    explanation = (f"The dominant detected emotion is '{dominant_emotion}', " f"which maps to the behavioral state '{mapped_emotion}'.\n" f"For the selected scenario '{scenario}' with difficulty level '{difficulty}', " f"the predicted decision tendency is '{decision}'")
    return {"Predicted Decision" : decision, "Confidence" : round(float(confidence), 3), "Detected Emotion" : dominant_emotion, "Mapped Emotion" :

```

```

mapped_emotion, "Explanation" : explanation}
def behavioral_interpretation(mapped_emotion, difficulty, decision):
    interpretation = []
    if mapped_emotion in ["anxious", "stressed", "panic", "overwhelmed"]:
        interpretation.append("Feeling anxious in challenging situations can
increase the tendency to avoid decisions.")
    elif mapped_emotion in ["calm", "focused"]:
        interpretation.append("A calm emotional state usually supports clear and
balanced decision-making.")
    elif mapped_emotion in ["concerned", "thinking"]:
        interpretation.append("A reflective emotional state may lead to careful
and cautious decisions.")
    else:
        interpretation.append(f"The emotional state '{mapped_emotion}' influences
how decisions are made under situational demands.")
    if difficulty in ["high", "very high"]:
        interpretation.append("Higher difficulty levels can increase emotional
pressure and effect decision patterns.")
    elif difficulty == "medium":
        interpretation.append("Moderate difficulty allows better control over
emotional reactions.")
    else:
        interpretation.append("Lower difficulty situations reduce emotional
pressure on decisions.")
    if decision == "avoidant":
        interpretation.append("Using structured planning and breaking the problem
into smaller steps may help reduce avoidance.")
    elif decision == "impulsive":
        interpretation.append("Taking a short pause before deciding may improve
decision quality.")
    elif decision == "analytical":
        interpretation.append("Continuing structured analysis can support
consistent decision outcomes.")
    elif decision == "cautious":
        interpretation.append("Balancing caution with timely action may improve
results.")
    else:

```



```

        interpretation.append("Improving emotional awareness may enhance decision
        effectiveness.")
    return " ".join(interpretation)

```

J. Streamlit User Interface

```

import sys
import os
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))
import plotly.graph_objects as go
import plotly.express as px
import pandas as pd
import streamlit as st
from core.simulation_engine import simulate_behavior, behavioral_interpretation
from core.simulation_engine import scenarios, difficulty_map
st.set_page_config(page_title = "HDT - Behavior Simulation", layout = "wide")
st.markdown(""" <style> body {background-color: #f5f7fa;} h1 {color: #1f2937;} h2
    , h3 {color: #374151;} .stButton > button {background-color: #2563eb; color:
    white; border-radius: 8px; height: 45px; font-size: 16px;} </style> """,
    unsafe_allow_html = True)
st.title("Human Digital Twin for Behavior Simulation Using AI")
st.markdown("### HDT System Interface")
st.divider()
st.subheader("Enter Situation Details")
user_text = st.text_area("Describe your situation:", height = 120)
selected_scenario = st.selectbox("Select Scenario:", scenarios)
selected_difficulty = st.selectbox("Select Difficulty Level:", difficulty_map[
    selected_scenario])
st.divider()
col1, col2, col3 = st.columns([2, 1, 2])
with col2:
    run_button = st.button("Run Simulation")
if run_button:
    if user_text.strip() == "":
        st.warning("Please enter a situation description.")
    else:
        result = simulate_behavior(user_text = user_text, scenario =
        selected_scenario, difficulty = selected_difficulty)

```

```

        interpretation_text = behavioral_interpretation(mapped_emotion = result["
Mapped Emotion"], difficulty = selected_difficulty, decision = result["
Predicted Decision"])
    st.divider()
    st.markdown("## Behavioral Simulation Result")
    col1, col2 = st.columns(2)
    with col1:
        st.metric("Predicted Decision", result["Predicted Decision"])
        st.metric("Detected Emotion", result["Detected Emotion"])
    with col2:
        st.metric("Mapped Emotion", result["Mapped Emotion"])
        confidence_percent = round(result["Confidence"] * 100, 1)
        st.metric("Confidence Level", f"{confidence_percent}%")
    st.divider()
    st.subheader("Emotion Confidence Visualization")
    emotion_df = pd.DataFrame({"Emotion": [result["Detected Emotion"]], "
Confidence": [result["Confidence"]]})
    fig_emotion = px.bar(emotion_df, x = "Emotion", y = "Confidence", color =
"Emotion", text = "Confidence", title = "Detected emotion confidence level")
    fig_emotion.update_traces(textposition = "outside")
    fig_emotion.update_layout(yaxis_range = [0, 1.1], height = 500, width =
500)
    col1, col2, col3 = st.columns([2, 3, 2])
    with col2:
        st.plotly_chart(fig_emotion)
    st.divider()
    st.subheader("Predicted Decision Tendency")
    decision = result["Predicted Decision"]
    decision_df = pd.DataFrame({"Decision": [decision], "Confidence": [result
["Confidence"]], "Label": [f"{confidence_percent}%"]})
    fig_decision = px.bar(decision_df, x = "Decision", y = "Confidence",
color = "Decision", text = "Confidence", title = "Predicted Behavioral
Decision")
    fig_decision.update_traces(textposition = "outside")
    fig_decision.update_layout(yaxis_range = [0, 1.1], height = 500, width =
500)
    col1, col2, col3 = st.columns([2, 3, 2])
    with col2:

```

```

        st.plotly_chart(fig_decision)
    st.divider()
    st.subheader("Emotion to Behavior Mapping")
    st.markdown(f"""
        **Detected Emotion:** {result["Detected Emotion"]}
        **Mapped Behavioral State:** {result["Mapped Emotion"]}
        **Predicted Decision Tendency:** {result["Predicted
Decision"]} """)
    st.divider()
    st.subheader("Explanation")
    st.write(result["Explanation"])
    st.divider()
    st.subheader("Behavioral Interpretation")
    st.write(interpretation_text)

```

K. Model Evaluation

```

import sys
import os
sys.path.append(os.path.abspath(".."))
import numpy as np
import pandas as pd
import joblib
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score, classification_report,
    confusion_matrix
from core.simulation_engine import simulate_behavior, behavioral_interpretation
pd.set_option("display.max_columns", None)
pd.set_option("display.max_colwidth", None)
print("Libraries loaded successfully")
print("="*50)
print("SYNTHETIC MODEL EVALUATION")
print("="*50)
synthetic_model = joblib.load("../models/synthetic_model.pkl")
X_syn = np.load("../data/synthetic_features_scaled.npy")
y_syn = np.load("../data/synthetic_labels_final.npy", allow_pickle = True)
print(X_syn.shape, y_syn.shape)
y_pred = synthetic_model.predict(X_syn)

```

```

print("Accuracy:", accuracy_score(y_syn, y_pred))
print("\nClassification Report:")
print(classification_report(y_syn, y_pred, zero_division = 0))
cm = confusion_matrix(y_syn, y_pred)
plt.figure(figsize = (8, 6))
sns.heatmap(cm, annot = True, fmt = "d", cmap = "Blues")
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
test_cases = [
    {
        "text" : "I have a new job opportunity but I feel nervous and unsure",
        "scenario" : "career",
        "difficulty" : "high"
    },

    {
        "text" : "I got a promotion and I feel confident about handling it",
        "scenario" : "career",
        "difficulty" : "medium"
    },

    {
        "text" : "I am facing financial issues and I feel stressed about money",
        "scenario" : "money",
        "difficulty" : "high"
    },

    {
        "text" : "I had an argument with my friend and I feel confused and upset",
        "scenario" : "relationship",
        "difficulty" : "medium"
    },

```

```

{
    "text" : "I am worried about my health and I feel anxious about it",
    "scenario" : "health",
    "difficulty" : "high"
},

{
    "text" : "I need to make a risky decision but I feel unsure and hesitant
",
    "scenario" : "risk",
    "difficulty" : "high"
}]
results = []
for case in test_cases:
    result = simulate_behavior(user_text = case["text"], scenario = case["
scenario"], difficulty = case["difficulty"])
    results.append({"Input" : case["text"],
                    "Scenario" : case["scenario"],
                    "Difficulty" : case["difficulty"],
                    "Detected Emotion" : result["Detected Emotion"],
                    "Mapped State" : result["Mapped Emotion"],
                    "Decision" : result["Predicted Decision"]
                    })
pd.DataFrame(results)
result = simulate_behavior(user_text = "I have a new job opportunity but I feel
nervous and unsure", scenario = "career", difficulty = "high")
result["Behavioral Interpretation"] = behavioral_interpretation(result["Mapped
Emotion"], "high", result["Predicted Decision"])
print("Case Study Result:\n")
print(f"Predicted Decision : {result['Predicted Decision']}")
print(f"Confidence Level : {result['Confidence']}")
print(f"Detected Emotion : {result['Detected Emotion']}")
print(f"Mapped Emotion : {result['Mapped Emotion']}")
print("\nExplanation:\n")
print(result["Explanation"])

```

```

print("\nBehavioral Interpretation:\n")
print(result["Behavioral Interpretation"])
print("="*50)
print("GOEMOTIONS MODEL EVALUATION")
print("="*50)
go_model = joblib.load("../models/goemotions_model.pkl")
X_go = joblib.load("../data/goemotions_tfidf_features.pkl")
y_go = np.load("../data/goemotions_labels.npy")
y_pred_go = go_model.predict(X_go)
print("Accuracy:", accuracy_score(y_go, y_pred_go))
print("\nClassification Report:")
print(classification_report(y_go, y_pred_go, zero_division = 0))
y_go_single = np.argmax(y_go, axis = 1)
y_pred_go = np.argmax(y_pred_go, axis = 1)
cm_go = confusion_matrix(y_go_single, y_pred_go)
plt.figure(figsize = (10, 8))
sns.heatmap(cm_go, cmap = "Blues")
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

```